Prayog — Costing Document

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. Summary

Cost block	Basis	Estimate
Development effort to date (1 Sep – 6 Nov 2026)	≈1,020 person-hours × ₹800/h loaded rate	**≈ ₹8.2 lakh**
Remaining effort (Sprint 3, demonstration)	≈100 person-hours	≈ ₹0.8 lakh
Tooling during development	GitHub, Jira, GitHub Actions on their free tiers	₹0 cash
Running cost, institute deployment (IITK VM, donated GPU nodes, Steward on the heuristic adapter)	section 4, option A	**≈ ₹0 cash / month** (≈ ₹6,000 / month opportunity cost of the VM and admin time)
Running cost, cloud deployment with the Steward on Claude Opus 5.5	section 4, option B + section 5	**≈ US$ 210 / month** at the assumed load
Running cost, single cloud VM with Docker Compose, Steward on the heuristic adapter	section 4, option C	**≈ US$ 40 / month**
Effort the same system would take without AI assistance (COCOMO II, organic)	23.3 KSLOC	≈ 65 person-months — about ten times the effort actually spent (section 3)

Every figure below states its assumption. Effort figures are capacity-based (5 members × 20 h/week, the figure the sprint plan was built on): Jira has no time tracking, so there are no timesheets to sum.

2. Development effort

2.1 Assumptions

Assumption	Value	Source
Team	5 members (Adithya, Areen, Disha, Aditi, Nideesh)	Sprint plan section 1
Effort per member	20 h / week	Sprint plan section 4
Hours per story point	2 h	Sprint plan section 4 (anchor: "register an account with a hashed password" = 3 points)
Loaded rate	₹800 / h	Entry-level software engineer at ≈ ₹12 lakh/year (≈ ₹625/h over 1,920 h) plus ≈ 30% overhead for equipment, office and benefits. A teaching-assistant rate (₹300/h) would cut every effort figure by 62%.

2.2 Effort by phase

Phase	Dates	Weeks	Person-hours	Cost (₹)
Requirements and architecture (D1)	1 Sep – 27 Sep	4.0	400	3,20,000
Sprint 1 — implementation, Agentic AI (D2)	28 Sep – 15 Oct	2.6	260	2,08,000
Sprint 2 — integration, web app, testing, security, performance (D3)	16 Oct – 6 Nov	3.1	310	2,48,000
D1 viva and D2 evaluation preparation	10 Oct, 15 Oct	—	50	40,000
Total to date			**≈ 1,020**	**≈ 8,16,000**
Sprint 3 — demonstration rehearsal (planned)	7 – 13 Nov	1.0	100	80,000
Total at completion			**≈ 1,120**	**≈ 8,96,000**

2.3 Effort by epic (story points)

Planned points come from `PRAYOG_backlog_jira_import.csv`; "Done" is the status in the Jira export. One point = 2 h.

Epic	Planned points	Done	Hours (planned × 2)	Cost (₹)
EP-01 Platform foundations	25	25	50	40,000
EP-02 Identity and Access	42	42	84	67,200
EP-03 Project and Workspace	8	8	16	12,800
EP-04 GPU Scheduling Core	43	43	86	68,800
EP-05 Execution and Job Lifecycle	30	30	60	48,000
EP-06 Results and Synchronization	8	8	16	12,800
EP-07 Node Operations and Maintenance	29	29	58	46,400
EP-08 Experiment Ledger	23	23	46	36,800
EP-09 Verified Reproduction	15	15	30	24,000
EP-10 Steward — Agentic AI	38	38	76	60,800
EP-11 Quality Engineering	39	0 in the 16 Oct export (this deliverable)	78	62,400
EP-12 Deliverables and Traceability	23	2	46	36,800
Total	**323**	**263**	**646**	**5,16,800**

The backlog's 646 hours cover only the sprint work; the 1,020 hours of section 2.2 also cover the requirements phase, reviews, integration that was not ticketed (the web application, the CLI rebuild fix, CI repairs) and the deliverable documents. The difference — about a third of all effort — is the untracked work the retrospective should make visible (Project Report section 6).

3. Size-based cross-check (COCOMO II, organic mode)

Measure (non-blank lines, `main` at `18767aa`)	Lines
Backend application code (`backend/`, without tests and migrations)	12,867
Steward package (`steward/src`)	6,286
CLI (`cli/src`)	1,548
Web application (JS/CSS/HTML in `backend/web` and the Steward UI)	2,571
Delivered source	**23,272**
Migrations (generated, excluded)	1,397
Tests (excluded; includes the Steward's 222-test suite kept outside `main`)	10,713

Effort = 2.4 × KSLOC^1.05 = 2.4 × 23.3^1.05 ≈ **65 person-months**; schedule = 2.5 × E^0.38 ≈ **12 months**. The team delivered it in ≈ 6.7 person-months (1,020 h ÷ 152 h/month) over 9.5 weeks.

The factor of ten does not mean COCOMO is wrong for traditional teams. It measures how much of the code was drafted with AI assistance (AI Engineering Log): what the team spent time on was specification, review, integration and testing, not typing. The honest reading for costing future work is that **review and verification, not code volume, are the cost driver** — D3's own defects (D3-03 to D3-08) were found by testing code that already "worked", which is where the remaining hours went.

4. Infrastructure running cost

Load assumption for both options: 300 registered users, 3,000 GPU jobs a month, 15% of runs failing, 200 GB of artifacts, 8 lab-owned GPU nodes.

Option A — institute deployment (recommended)

Item	Provider	Monthly cash cost	Note
API, scheduler tick, Steward worker and Runtime	IITK Computer Centre VM, 4 vCPU / 8 GB	₹0	opportunity cost ≈ ₹2,500/month at commercial VM prices
PostgreSQL 16	same VM or the CC's database service	₹0	
Artifact storage	500 GB on the CC's network storage	₹0	
GPU nodes	lab-owned, donated through the sharing model	₹0	electricity is already paid by the labs; Prayog only raises their utilisation
TLS, DNS	IITK proxy and `iitk.ac.in` subdomain	₹0	
Operations	≈ 4 h/month of an admin's time	≈ ₹3,200	patching, backups, upgrades (Deployment section 6–7)
Steward reasoning	heuristic adapter	₹0	section 5 for the Claude-backed variant

Option B — public cloud (for comparison; list prices to verify before purchase)

Item	Sizing	≈ US$ / month
Compute for API + tick + worker	2 vCPU / 4 GB instance, on-demand	35
Managed PostgreSQL	2 vCPU / 4 GB, 50 GB storage, automated backups	60
Object storage	200 GB + requests + egress	10
Load balancer / TLS	one application load balancer	20
Monitoring and logs	basic tier	5
Subtotal (without GPUs and without Claude)		**≈ 130**

Option C — one cloud VM with Docker Compose (the deployment this package verifies locally)

`docker-compose.deploy.yml` runs PostgreSQL, the API and the tick on a single host. It is the cheapest way to meet the course's cloud deployment requirement, and it needs no managed database or load balancer.

Item	Sizing	≈ US$ / month	Source
AWS EC2 t3.medium, Mumbai (`ap-south-1`)	2 vCPU / 4 GB, on-demand at about $0.0448/hour × 730 h	33	third-party price listings (DoiT, aws-pricing.com); check the AWS page before buying
EBS gp3 disk	30 GB at about $0.08–0.10/GB-month	3	the $0.08 figure is for `us-east-1`; no confirmed Mumbai price found
Public IPv4 address and data transfer	one address, light traffic	4	estimate
Total		**≈ 40**	

Against the budget. New AWS accounts now get up to $200 in credits rather than the older 12-month free tier, so a VM for the two-week demo window costs about $20 of credit. Sources disagree on how long the credits last (6 or 12 months); check before relying on them. Stopping the VM between sessions stops the compute charge.

Estimated against actual. Actual cloud spend to date is ₹0, because the platform has not yet been deployed to a cloud account (Deployment section 9). Measured load (Performance report) came from a 12-core laptop, so it does not prove a t3.medium can carry the same peak; the design workload of 200–400 jobs a day is far below what was measured, but a t3.medium run should be measured once it exists.

GPU capacity is deliberately absent from options B and C: renting the eight GPUs the labs already own would cost far more than everything else combined, and avoiding that is the reason Prayog exists.

5. Steward AI (Claude API) cost

Prices are Anthropic's first-party list prices per million tokens (cached 6 Oct 2026): Claude Opus 5.5 $4 input / $20 output / $0.20 cache read; Claude Sonnet 5.5 $2 / $10 / $0.20; Claude Haiku 5.5 $0.10 / $0.50. Token sizes per call are estimates from the Steward's context limits (`steward/DESIGN.md`), not measurements: no live Claude call has been made yet.

Steward context	Calls / month	Input tokens / call	Output tokens / call (incl. thinking)	Opus 5.5 $/month	Sonnet 5.5 $/month	Haiku 5.5 $/month
S1 run-failure diagnosis	450 (15% of 3,000 runs)	12,000	3,000	48.6	24.3	1.2
S2 hardware correlation	30	6,000	1,000	1.3	0.7	0.03
S3 diagnostic chat (5 turns)	100 sessions	5 × 15,000	5 × 1,000	40.0	20.0	1.0
S4 optimisation advice	100	20,000	2,000	12.0	6.0	0.3
Total				**≈ 102**	**≈ 51**	**≈ 2.5**

With prompt caching of the fixed system prompt and knowledge-base chunks (about half of each input) at the cache-read price, the Opus 5.5 total falls to ≈ $75/month. Cost scales linearly with the failure rate: a course with many failing first runs could triple S1. The output guard and the per-user rate limit (FR-STW) bound the worst case; a monthly spend limit on the API key is the operational control.

Recommendation: start on the heuristic adapter (₹0), switch S1 to Claude Sonnet 5.5 once the evaluation set (`steward/eval/`) shows it beats the heuristic, and keep Opus 5.5 for S3/S4 only if the evaluation shows a measurable difference.

6. Cost risks

Risk	Effect	Mitigation
Failure rate higher than 15%	Steward cost rises linearly	per-user rate limit; Sonnet/Haiku for S1; monthly spend cap
Artifact growth (checkpoints)	storage cost and backup time grow	content-addressed deduplication already avoids storing identical files twice; retention policy for INCOMPLETE snapshots
Untracked effort	estimates for future work too low	log all work in Jira, including integration and documents